LEARNING MANAGEMENT SYSTEM

MODERATED LMS — FINAL PROJECT REPORT

Student: **Realeboha Makhaleme** Student No.: **202322669** Submission date: 7 November 2026

GitHub repository: https://github.com/realebohamakhaleme68-arch/CS4433 (final project: **master** branch)

1. Title page, abstract and table of contents

Abstract. This report documents the full life cycle of a Moderated Learning Management System (LMS) built for a secondary school as the CS4433 final project. The system lets administrators, lecturers, Heads of Department (HoDs) and students manage courses, assignments, submissions, marking and moderated release of marks, while recording every mark change in an audit log. The work followed the Waterfall model with V-Model verification and validation. It covers planning, requirements engineering, system design, a C++17 and SQLite implementation, automated testing, configuration management and quality assurance, deployment, maintenance (change request CR1) and professional issues. The main results are a layered architecture with five design patterns, a database in Third Normal Form, a 14 functional and 7 non-functional requirement baseline, 19 of 19 automated tests passing, 76.83% line coverage of the business-logic layer and zero static-analysis warnings. Evidence that can only be produced on GitHub or on school hardware is listed honestly as outstanding.

1 Title page, abstract, contents 1
2 Introduction 1
3 Project management plan 2
4 Requirements engineering 3
5 Software design document 4
6 Implementation 6
7 Verification and validation 6
8 Configuration mgmt and QA 7
9 Deployment 8
10 Maintenance (CR1) 8
11 Professional, legal, ethical 8
12 Conclusion 9
13 References 9
14 Appendices A to D 9

2. Introduction

Purpose. A secondary school needs a system to manage courses, assignments and assessment for four kinds of user: students, lecturers, HoDs and a system administrator. Every user logs in with a username and password and may only do what their role allows. Each submission passes through a defined life cycle (submitted, late, marked, moderated, released), and the school must be able to see who changed a mark, when and why. Online payment, video conferencing, automated grading, plagiarism detection and web or mobile interfaces are out of scope. The system is a C++17 console application that runs offline on one school computer or local network.

Repository and branches. All artefacts are in one GitHub repository: https://github.com/realebohamakhaleme68-arch/CS4433. The repository has **two branches**. **The final project is in the master branch**; the second branch is not the submitted version and should be ignored when marking. The master branch holds the source code (*src/*), the database schema (*db/schema.sql*), the automated tests (*tests/*), the build files (*CMakeLists.txt* and the Windows batch scripts), the GitHub Actions workflows (*.github/workflows/*), the project documents (*docs/*) and the raw test and analysis evidence (*test-results/*).

How this report was produced. This report merges six project documents: the project plan and management document, requirements engineering, the system design document, the test plan, the Phase 5 verification and validation report and the Phase 6 configuration management and quality assurance report. Only the important content is kept; the full documents and the source are in the repository. The work breakdown structure and risk register were drafted with DeepSeek, the use case specifications were drafted with Claude, and both were reviewed and edited by the student (see References). The sections follow the order required by the brief, and the content is grouped by the project phases of the Waterfall model.

Development approach. The project followed the Waterfall model (Sommerville, 2016): each phase ended with a documented deliverable, and implementation did not start before the design was baselined (Gate 2). The V-Model pairs each development artefact with a test level: requirements with acceptance tests, the SRS and use cases with system tests, the architecture with integration tests and detailed design with unit tests. Tools used are a UML tool, C++17 (GCC), CMake, SQLite, GoogleTest, Git and GitHub Actions, and Clang Static Analyzer.

3. Project management plan

3.1 Feasibility study

Dimension	Assessment	Verdict

Dimension	Assessment	Verdict
Technical	C++17 meets the object-oriented requirement; SQLite is embedded and serverless, which suits one school computer or LAN; CMake is cross-platform; GoogleTest and GitHub Actions are free and integrate with CI; cppcheck, clang-tidy or Clang Static Analyzer are command-line tools. Nothing needs a network at run time.	Feasible with standard, free, well-documented tools.
Operational	Users are not IT experts, so menus are role-filtered. Computers are old and the internet is unreliable, so the program is offline-first and lightweight. SQLite handles 800 students, and indexed queries make login under 2 seconds achievable.	Feasible if the interface stays simple and offline.
Economic	All tools are open source and the computers are already owned.	Zero licence or infrastructure cost.
Legal	Learner data (names, marks, submissions) must be protected, so password hashing and an audit log are mandatory. The Lesotho Data Protection Act (2011) applies. Accessibility (keyboard-only use, readable text output) and transparent late-penalty rules must be designed in.	Feasible if security, audit and fairness are designed in from the start.
Schedule	Seven weeks. Planning week 1; requirements weeks 1-2; design weeks 2-3 (high risk, Gate 2 blocks implementation); implementation weeks 4-5 (high risk, largest workload); V&V weeks 5-6; deployment week 6; maintenance and completion week 7.	Feasible only if Gate 2 design is finished by the end of week 3.

Recommendation: proceed. All five dimensions are satisfied. The main risks are schedule compression in weeks 4-5 and the conflicting stakeholder statements, both manageable through weekly milestones, early conflict resolution and strict use of the phase gates.

3.2 Work breakdown structure, schedule and critical path

Work package	Activities (WBS level 2)	Weeks
1.1 Planning and management	Feasibility study, WBS and Gantt chart, risk register, project management plan	1
1.2 Requirements engineering	Stakeholder analysis, conflict resolution, SRS, use case model with three fully dressed use cases (Gate 1)	1-2
1.3 System design	Architecture, class diagram with patterns, state, activity and sequence diagrams, ERD in 3NF, data dictionary, SQL schema (Gate 2)	2-3
1.4 Implementation	C++17 core and User hierarchy, business rules, SQLite data-access layer, security and audit, CMake and coding standard	4-5
1.5 Verification and validation	Test strategy, unit and integration tests (70% coverage target), system and acceptance tests, defect log (Gate 3)	5-6
1.6 Configuration and QA	Git branching and CI, static analysis, metrics, self code review	5-6
1.7 Deployment	Deployment diagram, installation guide, four user manuals, release v1.0	6
1.8 Maintenance	CR1 impact analysis, release v1.1, regression testing; final report and viva	7

The Gantt chart (in the project plan document) links the packages with dependencies. **Critical path:** stakeholder analysis, SRS, architecture, class diagram, behavioural models, Gate 2, core implementation, security/roles/audit, system and acceptance tests, Gate 3, release v1.0, CR1/v1.1 and the final report. A delay to any of these delays the final submission. Non-critical tasks (risk register, database design, CI set-up, test plan, unit tests) have one to three days of float. Milestones: Gate 1 (end of week 2), Gate 2 (end of week 3), Gate 3 and v1.0 (week 6), v1.1 and the viva (week 7). Report sections were written alongside each phase.

3.3 Risk register (initial)

ID	Type	Risk	Likelihood	Impact	Exposure	Mitigation	Owner
R1	Project	Gate 2 design not approved by end of week 3, blocking implementation	Medium	High	High	Freeze design scope early; daily design reviews in week 3; supervisor checkpoint mid-week	Self
R2	Project	Scope creep: features beyond the Must requirements	High	High	High	Prioritise in the SRS; change control for anything outside the baseline	Self
R3	People	Illness or emergency reduces available weeks	Low	High	Medium	Buffer in week 6; keep the repository always buildable	Self
R4	Technical	CI pipeline fails on the GitHub Actions configuration or old hardware	Medium	Medium	Medium	Test CI in week 4 with a minimal CMake project first	Self
R5	Technical	Data loss: no backup of the SQLite database or Git history	Low	High	Medium	Remote Git repository and a backup copy of the database	Self

ID	Type	Risk	Likelihood	Impact	Exposure	Mitigation	Owner
R6	People	Resubmission abuse or clock drift used to avoid lateness	Medium	Medium	Medium	Enforce timestamps in code; immutable timestamps; limit resubmission window	Self
R7	Project	Conflicting stakeholder statements (Principal against Lecturer on release)	High	Medium	High	Resolve in weeks 1-2 with an explicit conflict table; document decisions in the SRS	Self

The register is updated at each gate; the end-of-project status of each risk is in Appendix B.

4. Requirements engineering

4.1 Stakeholder analysis

Stakeholder	Interests
Principal	A simple system; marks checked by the HoD before students can see them.
Lecturer	Marks released as soon as marking is complete; ability to grant extensions.
Head of Department	See every mark in the department before release; 10% penalty per late day for three days, then zero.
Student representative	Correct and resubmit work before the deadline; written feedback; late work never simply rejected.
IT officer	Offline operation; quick response with 800 students; password protection.
Administrator / students	Manage accounts, courses and enrolment; view courses, submit work and read released marks.

4.2 Conflicts, ambiguities and their resolution

ID	Conflict or ambiguity	Resolution
C1	Principal: no mark before HoD checks. Lecturer: marks released immediately after marking.	Marks stay hidden until the HoD has reviewed them. The HoD has exactly 24 hours to approve, adjust or return a mark; if nothing is done, the mark is released automatically and audited as SYSTEM.
C2	Lecturer wants extensions; HoD wants strict late rules.	An extension moves the effective deadline for that student, so the work is not late. Every extension carries a reason and is permanently recorded.
C3	IT officer wants speed for 800 students on old computers.	Indexed queries and efficient data structures; measurable target of login within 2 seconds (NFR-1).
C4	Student representative wants resubmission; lecturer does not want endless re-marking.	Resubmission only before the effective deadline. After it, more time is possible only through a reasoned extension.
C5	Principal wants simplicity; IT officer wants strong security.	Security (hashing, parameterised SQL, audit) lives inside the code and is invisible to users; menus stay simple.
A1	“Simple” is undefined.	Role-filtered, menu-driven console with no unneeded options.
A2	“Password protected” is vague.	No plain text; passwords of at least 8 characters containing letters, digits and symbols, stored as salted hashes.
A3	What is a good reason for an extension?	Extensions are granted only by the lecturer with a written reason that is stored and can be inspected.
A4	When does each 10% deduction apply?	Each started 24 hours after the effective deadline counts as one day late (ceiling rule).

4.3 Software Requirements Specification (summary)

The SRS follows the structure of ISO/IEC/IEEE 29148 (Sommerville, 2016, ch. 4). Requirements have unique identifiers, MoSCoW priorities and a verification method. Functional requirements:

ID	Requirement	MoSCoW	ID	Requirement	MoSCoW
FR-1	Users log in with username and password	Must	FR-8	Late penalty 10% of the maximum mark per day for three days, zero after that	Must
FR-2	Role-based access: each role sees only permitted actions	Must	FR-9	Lecturer grants an individual extension (changes the deadline)	Should
FR-3	Admin creates, updates and deactivates accounts	Must	FR-10	Lecturer marks submissions and writes feedback	Must
FR-4	Admin manages departments and courses, assigns lecturers, enrols students	Must	FR-11	HoD moderates all marks in the department before release	Must
FR-5	Lecturer creates assignments with a deadline and maximum mark	Must	FR-12	HoD releases approved marks to students	Must
FR-6	Student views enrolled courses and assignments	Must	FR-13	Student views released marks and feedback	Must
FR-7	Student submits work (and may resubmit before the deadline)	Must	FR-14	Every mark change is audit-logged (who, when, old, new, reason)	Must

ID	Non-functional requirement (measurable)	MoSCoW	Verification
NFR-1	Login completes within 2 seconds with 800 user records stored	Must	Timed benchmark on target hardware
NFR-2	The system works fully offline (no network code)	Must	Inspection; run with network disabled
NFR-3	Passwords are stored only as salted hashes	Must	Unit test; database inspection
NFR-4	All database queries use parameterised statements	Must	Code review; injection test strings
NFR-5	Invalid input or database errors never crash the program	Must	Negative tests; transaction rollback test

ID	Non-functional requirement (measurable)	MoSCoW	Verification
NFR-6	Usable by non-IT staff through role-filtered menus	Should	Acceptance walk-through with teachers
NFR-7	Runs on old and new computers (portable C++17, no OS-specific code)	Must	Build on Windows and Linux

4.4 Use case model

The use case diagram covers all four actors. Student: Submit Assignment («include» Check Deadline), specialised by Resubmit Assignment, View Released Marks. Lecturer: Mark Submission and Write Feedback («include» Record Mark Change, «extend» Apply Late Penalty), Grant Extension. HoD: Moderate Marks («include» Record Mark Change, «extend» Release Marks). Administrator: manage users, departments, courses and enrolment. Three use cases are fully dressed in the requirements document: UC-01 Submit Assignment, UC-02 Mark Submission and Write Feedback, UC-03 Moderate Marks. UC-01 is summarised below.

UC-01 item	Specification
Preconditions	Student is logged in and enrolled; the assignment is published.
Main flow	(1) Select course and assignment. (2) System shows the effective deadline (deadline plus any personal extension) and current status. (3) Student chooses a file and submits. (4) System validates the file (not empty, allowed type, up to 10 MB). (5) «include» Check Deadline. (6) System saves the submission with a timestamp, status SUBMITTED. (7) Confirmation receipt.
Alternative flows	A1 resubmission before the deadline keeps the old version in history. A2 late by 1-3 days: status LATE, warning of the 10% per day penalty, confirmation requested. A3 late by more than 3 days: warning that the mark will be zero; work still accepted. A4 student cancels, nothing saved.
Exception flows	E1 invalid file: error, nothing saved. E2 not enrolled or not published: access denied. E3 resubmission after the deadline or marking: rejected, existing submission kept. E4 database error: transaction rolled back, previous submission unchanged.
Postconditions	Success: exactly one current submission with a timestamp and status SUBMITTED or LATE. Failure: no data changed.
Business rules	BR1 resubmit only before the effective deadline. BR2 effective deadline = deadline + extension. BR3 each started 24 hours late is one day; 10% per day for three days, then zero (applied at marking). BR4 late work is never rejected. BR5 only enrolled students may submit.

UC-02 adds a validated mark (0 to maximum), written feedback, an audit entry written in the same transaction as the mark, and a reason for any re-mark. UC-03 lets the HoD approve, adjust (with reason) or return a mark, and release only when every submission in the course is moderated.

5. Software design document

5.1 Architecture and design rationale

A **layered architecture** with four tiers was selected; dependencies point downwards only. The component diagram (Figure 1 of the design document) shows the components with provided and required interfaces.

Tier	Content	Why it exists
1 Presentation	Console UI with role menus and thin controllers	Collects input and shows results; holds no business rule.
2 Business logic	Application services, domain model, security services, audit logger, Strategy, Factory and Observer	All rules live here: RBAC, deadlines, late penalty, resubmission, moderation, audit.
3 Data access	Repository interfaces and SQLite implementations	The only place with SQL, always parameterised.
4 Persistence	One local SQLite file, *lms.db*	Durable offline storage on the school computer.

Why layered: it supports offline operation (one executable and one file, no server); the penalty, moderation and audit rules can be tested without a console; password hashing and all SQL sit in single places that can be reviewed separately; a menu console is enough for non-IT users; and the layers can be built and tested one at a time. **Rejected alternative 1, MVC:** in a console program business rules drift into the controllers, where they cannot be unit tested without driving the console, endangering the 70% coverage target; MVC pays off only when several views must stay in step with one model, and this system has one console view; MVC also says nothing about the data-access boundary the brief requires. Thin controllers are still kept in the presentation tier, so the useful part of MVC survives. **Rejected alternative 2, client-server with a network database:** the IT officer reports unreliable internet and old computers, and a network design adds latency, failure points, deployment effort and attack surface. The accepted trade-off is some forwarding code between tiers, kept small by thin services.

5.2 Class diagram, patterns and SOLID

The class model is split into five views (users and organisation; assessment model; design patterns; application services; data access) so that every attribute, visibility, type and multiplicity stays readable. *User* is abstract with pure virtual *roleName()* and *permissions()*, overridden by Student, Lecturer, HeadOfDepartment and Administrator (polymorphism). Submission and Mark are separate classes (1 to 0..1) because a mark exists only after marking and the audit log must reference a specific mark. Every service method takes the current Session first and begins with an authorisation check.

Pattern	Classes	Why chosen	Alternative rejected
Strategy	ILatePenaltyStrategy, StandardDeductionStrategy, NoPenaltyStrategy	The penalty rule is the rule most likely to change; a new rule is a new class and MarkingService is not edited.	if/else inside MarkingService mixes the rule with workflow and is harder to test.
Repository / DAO	I*Repository, SQLite*Repository	Services depend on interfaces and SQLite stays inside one tier.	SQL inside services breaks security review and testability.
Factory	UserFactory	The switch on role exists in one place; a new role changes one class.	Scattered *new Student(...)* calls.
Observer	IMarkingObserver, HoDNotifier	The HoD must see that a mark awaits moderation and its 24-hour window; MarkingService knows only the interface.	Calling the UI from the business layer couples the layers.

Pattern	Classes	Why chosen	Alternative rejected
Singleton	DatabaseConnection, SessionManager	One database handle and one session per running program.	Global variables with no controlled lifetime.

SOLID principle	How the design applies it
Single responsibility	PasswordHasher only hashes; AuditLogger only logs; AuthorizationService only checks permissions; each repository handles one aggregate.
Open/closed	New penalty rule = new strategy class; new role = new User subclass plus one line in UserFactory.
Liskov substitution	Any User subclass works where User& is expected; any SQLite repository can replace its interface.
Interface segregation	Six small repository interfaces; IMarkingObserver has a single method.
Dependency inversion	Services depend on interfaces, never on SQLite classes or concrete strategies; concrete objects are wired in *main()*.

5.3 Behavioural models

The **state machine** for the Submission life cycle has no Draft state because the case study has no drafts. The activity diagram uses swimlanes (Student, Lecturer, HoD, System) with decisions “Is submission late?” and “HoD approves?” and a dashed 24-hour automatic release. Two sequence diagrams cover “Submit assignment” (main path plus resubmission, late and closed-resubmission fragments) and “Moderate and release marks”.

From	Event [guard] / action	To
(start)	submit [now ≤ effective deadline]	Submitted
(start)	submit [now > effective deadline] / daysLate = ceil(delay / 24 h)	Late
Submitted	resubmit [now ≤ effective deadline] / version + 1, archive old file	Submitted (self-loop)
Submitted, Late	lecturer marks / save mark, apply penalty, audit, start 24-hour review window	Marked
Marked	HoD rejects with comment / lecturer re-marks, audit, window restarts	Marked (self-loop)
Marked	HoD approves or adjusts / audit	Moderated
Moderated	HoD releases [all course marks moderated] / set released time	Released
Marked	no HoD action within 24 hours / auto-release, audited as SYSTEM	Released

Lateness rule. DaysLate = 0 if submittedAt ≤ effectiveDeadline, otherwise ceil((submittedAt − effectiveDeadline) / 24 h). Penalty = 10% of the maximum mark per day for days 1 to 3; day 4 or more gives zero. Final mark = max(0, raw − penalty). With a maximum of 100 and a raw mark of 80, a submission exactly at the deadline scores 80, one second late scores 70, 24 h and 1 s late scores 60, 48 h and 1 s late scores 50, and 72 h and 1 s late scores 0.

5.4 Database design

The ERD (crow's foot) has nine tables, all in at least Third Normal Form: **Departments, Users, Courses, Enrolments, Assignments, Extensions, Submissions, Marks, AuditLog**. Dates are stored as UTC ISO-8601 text and converted to local time (Africa/Maseru, UTC+2) only for entry and display, so late days never depend on the computer's time zone setting.

Table	Key and main columns	Constraints and normalisation note
Users	user_id PK; username, password_hash, full_name, role, department_id, is_active	username UNIQUE and indexed; role CHECK in (Admin, Lecturer, HoD, Student); the Argon2id string embeds the salt; Lecturer and HoD need a department; one HoD per department (partial unique index); accounts are deactivated, never deleted.
Courses, Enrolments	course_id PK, department_id FK, lecturer_id FK; enrolment_id PK	The lecturer's department is not copied into Courses, avoiding a transitive dependency. UNIQUE (student_id, course_id) resolves the many-to-many link.
Assignments, Extensions	assignment_id PK, course_id FK, deadline, max_mark; extension_id PK	max_mark CHECK > 0. Extensions are a separate append-only table (trigger) so that every extension is a permanent record.
Submissions	submission_id PK; file_path, submitted_at, version, days_late, status	UNIQUE (assignment_id, student_id); status CHECK in (Submitted, Late, Marked, Moderated, Released); mark and feedback moved out because they exist only after marking.
Marks	mark_id PK; submission_id UNIQUE; raw_mark, penalty_points, final_mark, feedback, review_due_at	final_mark is stored on purpose: after moderation it can differ from raw minus penalty, so it is not derivable and 3NF holds.
AuditLog	log_id PK; mark_id FK, user_id FK, action_type, old_value, new_value, reason, changed_at	ON DELETE RESTRICT keeps history; triggers make it append-only. User names are read through user_id, never copied.

Class-to-table mapping. The four User subclasses map to one Users table (single table inheritance with a role discriminator), which avoids joins at login; studentNo and employeeNo share reference_no. Enrolment becomes an association table; Student.enrolledCourses is a join, not a column. Session, services, strategies, factory, observers and repositories are behaviour only and have no table, and a session exists only in memory so that it disappears when the program closes. Enumerations are TEXT with CHECK constraints, and camel-case attributes map to snake-case columns. Indexes support the 800-student target: Courses(department_id), Enrolments(course_id), Assignments(course_id), Submissions(assignment_id, status), Marks(review_due_at, released_at), AuditLog(mark_id) and Extensions(assignment_id, student_id). A reserved SYSTEM user (id 1, inactive, hash that never verifies) is the actor for automatic releases.

6. Implementation

Structure. The code base has 34 C++ source and header files under *src/* (3,307 physical lines, 2,920 code lines), organised in the same four layers as the design: *util* (TimeUtil, PasswordHasher, InputValidator, Exceptions), *domain* (Models, the abstract User and its four subclasses, UserFactory), *data* (DatabaseConnection, Statement, I*Repository, SQLite*Repository) and *service* plus *ui* (Session, AuthenticationService, AuthorizationService, AuditLogger, penalty strategy, observer, the services, Console, controllers and Application). The build uses CMake with

C++17, compiler warnings on (*-Wall -Wextra -Wpedantic -Wshadow*) and the schema embedded in the executable, so *lms.db* is created automatically on first start. The documented coding standard (*docs/CODING_STANDARD.md*) fixes naming, layering (a layer never includes a header from a layer above), exceptions, RAII transactions, no raw *new*/*delete* and a *.clang-format* file.

Key design decisions. (1) Every public service method takes *const Session&* first and calls the authorisation service, so RBAC and department or ownership scope are checked in one place. (2) A mark change and its audit entry run in one RAII *Transaction*, so both succeed or both roll back. (3) The late-penalty rule is injected as an *ILatePenaltyStrategy*. (4) Expired HoD review windows are processed whenever the program starts or someone logs in, and the release is audited as SYSTEM. (5) Uploaded files are copied to *submissions/a<assignment>/s<student>/v<version>_<file>* so that earlier versions remain available.

Security. No plain-text password is stored. With libsodium available, passwords are hashed with Argon2id, a recognised memory-hard algorithm whose output embeds a random salt and its cost parameters; without libsodium the program uses a built-in salted SHA-256 hasher, and CMake and the start-up banner report which one is active. Password rules require at least 8 characters with letters, digits and symbols. All SQL goes through *Statement::bind*, so queries are parameterised, and an injection-style username is refused in tests. Failed logins return one generic message. Expected problems throw *ValidationException*, *AuthorizationException* or *DatabaseException* (all deriving from *AppException*), which controllers catch, so invalid input does not crash the program. The audit log and extensions tables reject UPDATE and DELETE through triggers.

Persistence. All data is in one SQLite file reached only through the repositories. A copy *lms.db.bak* is made at every start-up (risk R5). Foreign keys are enabled and constraints in the schema back up the validation in code.

Design said	Implementation did	Reason
UserFactory in the application services	Placed in *src/domain/*	The repository builds User objects; this keeps the data tier independent of the business tier.
Permission list without “view courses”	Added VIEW_COURSES for all roles	FR-6 needs a permission to check.
HoDNotifier as the notification route	In-memory counter; the HoD menu reads the database count	The notifier is lost when the program closes; the database is the source of truth.
UC-02 alternative A3 (save feedback only)	Not implemented	The Draft state was removed in design and Marks.raw_mark is NOT NULL.
Rule for a mark returned by the HoD not specified	Return to lecturer moves review_due_at 24 hours forward	Prevents a rejected mark being auto-released before re-marking.

Implementation limits. The 24-hour auto-release (conflict C1) relaxes the Principal's rule that no mark is seen before the HoD has checked it. The program trusts the computer clock offline, so a changed system clock could influence lateness (risk R6); immutable created-at values make this detectable but not preventable. A fully offline system cannot remove this weakness.

7. Verification and validation

7.1 Test strategy (V-Model)

Level	Artefact verified	Approach	Entry / exit criteria
Unit	Detailed design: domain classes, penalty strategy, time and input validation, user factory, password hashing	GoogleTest; equivalence partitioning and boundary value analysis	Entry: source compiles, interfaces baselined. Exit: critical unit tests pass, boundaries and invalid input covered, no critical defect.
Integration	Architecture: services with SQLite repositories, audit logger, transactions	GoogleTest workflow on in-memory SQLite	Entry: unit-tested components, schema initialises. Exit: service and repository tests pass; rollback and audit consistency verified.
System	SRS and use cases UC-01 to UC-03 including exception flows	Scenario tests for main, alternative and exception paths	Entry: integrated build. Exit: every path executed or marked not run; critical defects fixed.
Acceptance	Stakeholder goals and measurable NFRs	Checklist, user review, performance and hardware checks	Entry: release candidate and users available. Exit: criteria met and signed off, or deviations documented.

The test plan also specifies state-transition testing of the submission life cycle, a role-by-action matrix for RBAC, and negative testing with SQL-injection strings. Real repositories on an in-memory database were used instead of hand-written fakes; this is a documented deviation from the design rationale because it gives the same isolation at lower cost. Time is controlled by a clock override, so lateness tests are deterministic.

7.2 Test design

Input	Valid partition	Invalid partition and boundaries
Username	3-32 letters, digits, dot, underscore or hyphen	2 or 33 characters, spaces, quotes, SQL text
Password	8-128 characters with letters, digits and symbols	7 characters, letters only, digits only, no symbols
Mark	0 up to the maximum mark inclusive	-0.01, max+0.01, NaN, malformed text
Submission file	Existing, non-empty, allowed type, up to 10 MB	0 B, 10,485,761 B, disallowed extension, missing file
Review window	0 to 23 h 59 m 59 s after marking: no release	24 h 00 m 00 s: release

Submission time (deadline D)	D − 1 s	D	D + 1 s	D + 24 h	D + 24 h 1 s	D + 48 h 1 s	D + 72 h	D + 72 h 1 s
Days late	0	0	1	1	2	3	3	4
Penalty (maximum 100)	0	0	10	10	20	30	30	100 (final 0)

7.3 Results, coverage and defects

The source was rebuilt cleanly with GCC 14.2.0, CMake 3.31.6, SQLite 3.46.1 and GoogleTest 1.14.0 on Ubuntu. **All 19 CTest cases passed (100%)** with zero failed or skipped and no compiler warnings: 9 GoogleTest unit cases, 1 integration workflow, 8 deadline and date instances (seven parameterised deadlines and one invalid-date case) and the retained legacy test executable. The automated inventory is UT-01 deadline boundaries, UT-02 invalid dates, UT-03 and UT-04 penalty strategies, UT-05 and UT-06 input partitions and mark boundaries, UT-07 user factory and permissions, UT-08 salted hash, IT-01 full workflow, IT-02 transaction rollback and append-only records and IT-03 the 24-hour automatic release.

Source file (business logic)	Lines run / total	Coverage	Source file	Lines run / total	Coverage
domain/Models.cpp	28 / 45	62.22%	service/Security.cpp	23 / 29	79.31%
domain/UserFactory.cpp	12 / 13	92.31%	service/Services.cpp	304 / 394	77.16%
service/AuditLogger.cpp	11 / 11	100.00%	service/Session.cpp	10 / 13	76.92%
Total: 388 / 505 lines = 76.83% (target 70%)			Utilities: InputValidator 97.92%, PasswordHasher 95.29%, TimeUtil 96.77%		

System scenarios. ST-01 to ST-11 derive from the use cases and passed through the integration workflow: submit before the deadline, resubmit with version increment, late acceptance with days late recorded, resubmission after the deadline rejected, invalid file or non-student rejected, lecturer marks while the student sees nothing, invalid mark or re-mark without reason rejected, HoD approve, adjust and reject, release blocked while an unmoderated submission remains, released marks visible, and automatic release after 24 hours. **Defect log:** DEF-05-01 (High) – the Administrator menu option “Create a user” collected details but never called *AdministrationService::createUser* and fell through to “List users”. Cause: missing password prompt, missing service call and missing success message. It was found during compiler-warning review, fixed, and retested with a clean rebuild and 19 of 19 tests passing.

Outstanding (not claimed as passed). AT-01 teacher usability, AT-02 login under 2 seconds with 800 stored users (needs target hardware and an 800-user dataset), AT-03 offline run, AT-04 keyboard and screen-reader accessibility, the Argon2id branch (libsodium was unavailable, so only the SHA-256 fallback was tested), the hosted GitHub Actions result, and stakeholder sign-off. The planned test plan range of up to 40 unit cases is larger than the 19 executed cases, so remaining unit tests are future work.

8. Configuration management and quality assurance

Configuration management. The documented strategy is a feature-branch workflow: a stable trunk, short-lived *feature/**, *fix/** and *hotfix/** branches, pull-request review before merge, conventional commit messages (for example “ci: add build and test workflow for Phase 6”) and annotated release tags after a green build. The configuration document names the stable branch *main*; the submitted repository's final version is on **master**, and the CI workflows trigger on both names, so the pipeline is not affected. Two GitHub Actions workflows exist: *phase5-tests.yml* builds the project and runs CTest on each push and pull request, and *phase6-quality.yml* runs on pushes and pull requests and has two jobs, (1) CMake configure, build and CTest, and (2) Clang Static Analyzer over the implementation files with the log uploaded as an artefact.

Quality activity	Result
Static analysis	Clang Static Analyzer 17.0.0 in C++17 mode over all 15 *.cpp* files: 0 warnings and 0 errors; the GCC build also had no warnings. No production change was needed.
Regression run	Clean local build, CTest 19 of 19 passed in 18.26 seconds.
Metrics	34 files; 3,307 physical lines; 2,920 code lines; 85 comment-only lines; 302 blank lines. Estimated McCabe complexity of key methods: SubmissionService::submit 5, ModerationService::adjust 5, reject 3, release 4, processExpiredReviews 3, AdministrationService::createUser 7 and setActive 8 (the highest).
Self code review	Checklist covering build reproducibility, warnings, CI, static analysis, tests, authorisation, transactions, password handling, input validation, error paths, repository hygiene and documentation. Changes made: the CI workflow, analysis script and evidence, metrics script and report, branching and release documentation and the review record. No production behaviour changed.

Honest status. The Phase 6 package was prepared from a ZIP without Git metadata, so a pull request, a hosted Actions run, a downloaded analysis artefact and the annotated release tags must be captured from the GitHub repository and added to the evidence before the claim can be made complete. The Phase 6 document suggests the tag *v1.1.0*; the brief requires *v1.0* for the first release and *v1.1* for the change request, and tags should follow the brief.

9. Deployment

Deployment diagram. The UML deployment diagram has one node, the school computer (Windows 10 or later, or Linux), with the artefact *lms.exe* (or *lms*), the database artefact *lms.db* with its backup *lms.db.bak* and the *submissions/* folder. A local network deployment uses the same executable on a shared drive; there is no server and no internet link.

Installation guide (technician).

- Install MSYS2, open the UCRT64 terminal and install GCC, CMake, Ninja, SQLite and (optionally) libsodium with *pacman*; add *C:\msys64\ucrt64\bin* to PATH. On Ubuntu use *apt install build-essential cmake ninja-build libsqlite3-dev libsodium-dev*.
- Unzip or clone the **master** branch to *C:\lms*. Double-click *build.bat* (or *simple_build.bat* without CMake); it must end with *BUILD OK*.
- Double-click *run.bat* (or *simple_run.bat*). On first start the program asks for the first administrator; there is no default password.
- Run *test.bat* or *ctest --test-dir build --output-on-failure* to confirm the installation.

User manual	Main tasks (menu-driven console)
Administrator	Create the first administrator; create departments, users, courses; assign lecturers; enrol students; reset passwords; deactivate accounts.
Lecturer	Create assignments with a deadline and maximum mark; grant an extension with a reason; mark submissions with feedback (late penalty is applied automatically); re-mark with a reason.
Head of Department	Review marks of the department with raw mark, penalty and final mark; approve, adjust (with reason) or return; release; automatic release after 24 hours.
Student	View courses and assignments; submit and resubmit before the deadline; read the late warning; view released marks and feedback.

Release notes v1.0. Console application with four roles, SQLite persistence, late-penalty, extension, resubmission and moderation rules, 24-hour auto-release and a full audit log. Known limits: no Draft feedback, console interface only, clock trust, and acceptance tests still to run on school hardware. The tag *v1.0* has to be created on the final commit of **master**.

10. Maintenance (change request CR1)

CR1: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list.”

Classification. This is **perfective maintenance** (an enhancement that adds user-visible capability to a working system), not corrective (nothing is broken), adaptive (the environment is unchanged) or preventive. It is justified because the existing rule in FR-9 is kept and extended in scope.

Item type	Affected items
Requirements	FR-9 (extension) is extended to a class scope; FR-6 (students view assignments) must show extensions; FR-7 and FR-8 rely on the effective deadline and are indirectly affected; NFR-5 for validation of the new input.
Use cases	UC-01 Submit Assignment (step 2 and BR2, effective deadline); new use case Grant Extension to Class; View Assignments.
Diagrams	Use case diagram, class diagram view 2B and 2D (Extension, AssignmentService), sequence diagram for Submit assignment, ERD.
Classes and files	AssignmentService::grantExtension (add a class-scope method), findEffectiveDeadline, SubmissionService::listMyAssignments, ExtensionRepository, console controllers for the Lecturer and Student menus (*Services.cpp*, *SQLiteRepositories.cpp*, *Controllers.cpp*).
Tables	Extensions: keep one row per student (append-only audit) and insert one row for each enrolled student in a single transaction, optionally adding a batch identifier column; Assignments and Enrolments are read only.
Tests	Existing UT-01 (deadline boundaries), IT-01 (extension makes work on time) and ST-02 to ST-04 are re-run; new tests: class extension creates one row for every enrolled student, a non-enrolled student is unaffected, a non-owner lecturer is refused, an earlier date is rejected, and the extension appears in the student's list.

Implementation and regression testing. The change is implemented as release **v1.1** on a feature branch merged by pull request, followed by updates to the SRS, UC-01, the class and sequence diagrams and the ERD note. Regression testing re-runs all 19 existing CTest cases plus the new tests, and release is allowed only when the whole suite is green. **Status:** the repository documents supplied for this report cover phases up to 6; the v1.1 code, the new tests and their results must be taken from the repository (or added) before CR1 can be reported as delivered.

11. Professional, legal and ethical issues

Issue	Discussion and how the design addresses it
Data protection	Learner names, marks, feedback and submitted work are personal data, covered in Lesotho by the Data Protection Act (2011). The design stores the minimum data, restricts each role to what it needs (RBAC and department or ownership scope), keeps the database on the school computer with no network transfer, hashes passwords with salts and deactivates rather than deletes accounts so that history remains accurate. The school should restrict physical and operating-system access to *lms.db* and the backup.
Security of assessment records	Marks are hidden until released; every change is audited with who, when, old value, new value and reason in an append-only table written in the same transaction as the change; parameterised SQL prevents injection; generic login errors do not reveal which usernames exist. Remaining risks are the clock-trust weakness and unencrypted storage of the database file.
Accessibility	A plain-text, keyboard-only console is usable with screen readers and has no mouse or colour dependency, and menus are short and role-filtered. This has not yet been tested with an accessibility reviewer (AT-04), and a graphical interface would need a separate review.
Fairness of late penalties	The rule is simple, published and applied by the program, not by a person, so identical lateness gets an identical result. Extensions with a reason protect students with a good cause, and late work is never rejected. Ethical concerns are the cliff edge to zero after three days, and the one-second boundary of the ceiling rule; both are documented, tested and visible to the student in the late warning.
Professional practice	Requirements conflicts were resolved openly, the work followed a documented standard and phase gates, limits are stated and not hidden, and AI tools were used for drafts only and are acknowledged (ACM/IEEE Software Engineering Code of Ethics). The 24-hour auto-release is an ethical trade-off between assessment integrity and timeliness and is recorded for the supervisor.

12. Conclusion

The project delivered a working, offline Moderated LMS that follows its own requirements and design: a layered C++17 application with five design patterns, an audited SQLite database in Third Normal Form, role-based access for four roles and a late-penalty, extension, resubmission and moderation process that is tested at the boundaries. Verification gives 19 of 19 passing tests, 76.83% business-logic coverage and no static-analysis findings. Lessons learned: keeping rules in the service layer made them testable; resolving stakeholder conflicts in week 1 prevented rework; and artefacts must be kept consistent, for example the WBS says 12 functional and 6 non-functional requirements while the SRS has 14 and 7. Open work is listed honestly: hosted CI evidence, tags v1.0 and v1.1, the CR1 code and regression results, and acceptance tests on school hardware. Final submission is the PDF report and the **master** branch of the GitHub repository.

13. References

Anthropic (2026) *Claude* [AI assistant, used to draft the use case specifications]. Available at: https://claude.ai (Accessed: 3 November 2026).

DeepSeek (2026) *DeepSeek chat* [AI assistant, used to draft the work breakdown structure and risk register]. Available at: https://chat.deepseek.com/ (Accessed: 3 November 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn, Global edn. Harlow: Pearson Education Limited.

Sommerville, I. (2016) *Chapter 2: Software Processes* [Lecture slides]. Software Engineering, 10th edn. Pearson.

Sommerville, I. (2016) *Chapter 4: Requirements Engineering* [Lecture notes]. Software Engineering, 10th edn. Pearson.

Sommerville, I. (2016) *Requirements Engineering Process* [Lecture slides, PPTX], and *Introduction to Software Engineering* [Lecture slides]. Pearson.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering*. Geneva: ISO.

Google (2023) *GoogleTest* 1.14.0. Available at: https://github.com/google/googletest. SQLite Consortium (2024) *SQLite* 3.46.1. Available at: https://sqlite.org.

Project repository (2026) *CS4433*. Available at: https://github.com/realebohamakhaleme68-arch/CS4433 (master branch).

14. Appendices

Appendix A: Requirements traceability matrix

Req.	Use case	Design and code (classes, tables)	Tests
FR-1	Login	AuthenticationService::authenticate, Console, Application; Users	Wrong password, unknown user, SYSTEM refused
FR-2	All	User::permissions() per subclass, AuthorizationService (role and department, owner or lecturer scope)	Student, lecturer and other-department HoD refused
FR-3	Admin	AdministrationService (createUser, updateUser, resetPassword, setActive), UserFactory; Users	Create, duplicate, weak password, deactivate; UT-07
FR-4	Admin	AdministrationService; Departments, Courses, Enrolments	Duplicate department and enrolment
FR-5	Lecturer	AssignmentService::createAssignment; Assignments	Past deadline refused
FR-6	Student	SubmissionService::listMyCourses, listMyAssignments	Manual (acceptance)
FR-7	UC-01	SubmissionService::submit, Submission::canResubmit; Submissions	Version 2 before deadline; refused after; ST-01 to ST-05
FR-8	UC-02	ILatePenaltyStrategy, StandardDeductionStrategy, MarkingService; Marks	UT-01, UT-03, UT-04; boundary table
FR-9	UC-01	AssignmentService::grantExtension, findEffectiveDeadline; Extensions (append-only trigger)	Extension makes work on time; IT-01, IT-02
FR-10	UC-02	MarkingService::markSubmission; Marks	Raw, penalty, final; re-mark needs reason; ST-06, ST-07
FR-11	UC-03	ModerationService (approve, adjust, reject)	Approve, adjust, reject; other department refused; ST-08
FR-12	UC-03	ModerationService::release, processExpiredReviews	Release; 24-hour auto-release; ST-09, ST-11, IT-03
FR-13	Student	SubmissionService::listMyReleasedMarks	Hidden before release, visible after; ST-10
FR-14	UC-02, UC-03	AuditLogger inside Transaction; AuditLog and triggers	Who, when, old, new, reason; tamper blocked; IT-02
NFR-1 to 7	All	Unique index on username; no network code; PasswordHasher; Statement::bind; exception hierarchy; role menus; C++17	UT-08, injection test, rollback test; NFR-1, NFR-2 and NFR-6 pending AT-01 to AT-03

Appendix B: Updated risk register (end of project)

ID	Status	Update
R1	Closed	Design was completed before implementation started; no Gate 2 delay is recorded in the documents.
R2	Mitigated	Scope held to the Must requirements; UC-02 “save feedback only” was deliberately left out.
R3	Open	Remaining risk to v1.1 delivery and the viva; the repository must stay buildable.
R4	Open	Workflows are written and tested locally; the hosted GitHub Actions run is still to be captured.
R5	Mitigated	Remote repository plus *lms.db.bak* created at every start-up.
R6	Partly mitigated	Immutable created-at values and append-only audit; changing the system clock cannot be prevented offline.
R7	Closed	All conflicts resolved in the requirements document; the 24-hour auto-release needs supervisor confirmation.
R8 (new)	Open	Document inconsistencies (12 FR + 6 NFR in the WBS against 14 + 7 in the SRS; tag names; main against master) must be corrected before submission.

Appendix C: Test results summary

Metric	Result	Metric	Result
CTest tests	19 passed, 0 failed, 0 skipped (100%)	Business-logic coverage	76.83% (388 / 505 lines)
Build	Clean rebuild, no compiler warnings	Static analysis	0 warnings, 0 errors (15 files)
Defects	1 found (High), fixed and retested	Not run	AT-01 to AT-05 (users, hardware, accessibility)

Appendix D: Reproduction commands

cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug -DLMS_USE_LIBSODIUM=OFF | cmake --build build --parallel 2 | ctest --test-dir build --output-on-failure | ./scripts/run_static_analysis.sh | python3 scripts/phase6_metrics.py